

Laboratory 11

Zephyr RTOS: Port, Device Tree & Kconfig

Real-Time Operating Systems • M.Eng. ECE • KMUTNB

Platform	NXP FRDM-MCXN236 (ARM Cortex-M33 @ 150 MHz)
RTOS	FreeRTOS v10.6
Toolchain	ARM GNU Toolchain (arm-none-eabi-gcc) + Make
Estimated time	4–5 hours
CLO mapping	CLO 5 & CLO 6

1. Objectives

By completing this lab you will be able to:

1. **Set up** a Zephyr RTOS workspace for the FRDM-MCXN236 using `west`.
2. **Port** the Lab 03 producer-consumer pipeline from FreeRTOS to Zephyr threading APIs.
3. **Write** a Device Tree overlay to add a simulated BME280 sensor on I2C.
4. **Use** the Zephyr Sensor API (`sensor_sample_fetch` / `sensor_channel_get`) with a DT-bound driver.
5. **Compare** binary size, API surface, and configuration overhead between FreeRTOS and Zephyr.

2. Prerequisites

- ☐ Lab 03 producer-consumer source (for reference during porting).
- ☐ Python 3.8+ and `pip` available.
- ☐ At least 4 GB free disk space (Zephyr workspace + HALs).
- ☐ Git installed.
- ☐ USB connection to FRDM-MCXN236 (J-Link or MCU-Link).

Note

Time note: `west` update downloads ~1.5 GB of source. Do this on a fast network connection. The download is a one-time setup; subsequent builds are fast.

3. Background: Zephyr vs. FreeRTOS

Aspect	FreeRTOS	Zephyr
Configuration	<code>FreeRTOSConfig.h</code> (#define)	<code>prj.conf</code> (Kconfig)
Hardware desc.	Code (<code>GPIO_PinInit(...)</code>)	Device Tree (<code>.dts</code> / <code>.overlay</code>)
Build system	Make / CMake (manual)	<code>west</code> (unified)
Driver model	SDK-specific (<code>fsl_gpio.h</code>)	Unified API (<code>gpio.h</code>)
Networking/BT	External libraries	Built-in subsystems
Certification	Qualified kits available	Partial (Safety Cert. project)

4. Project Structure

```
lab11_zephyr/
  CMakeLists.txt
```

```

prj.conf          <- Kconfig (replaces FreeRTOSConfig.h)
app.overlay       <- Device Tree customisation
src/
  main.c          <- Zephyr entry, LOG_INF banner
  sensor_thread.c/.h <- producer (replaces vSensor1Task)
  logger_thread.c/.h <- consumer (replaces vLoggerTask)
boards/
  frdm_mcxn236.conf <- board-specific Kconfig overrides (optional)

```

5. Part A — Workspace Setup

5.1. Step 1 — Install west

```
pip install west
```

5.2. Step 2 — Initialise the Zephyr Workspace

```

mkdir ~/zephyrproject && cd ~/zephyrproject
west init -m https://github.com/zephyrproject-rtos/zephyr --mr v3.6.0
west update          # ~20 minutes on first run
west zephyr-export   # registers CMake package
pip install -r zephyr/scripts/requirements.txt

```

5.3. Step 3 — Verify with hello_world

```

cd ~/zephyrproject
west build -b frdm_mcxn236 zephyr/samples/hello_world
west flash

```

Expected terminal output:

```

*** Booting Zephyr OS build v3.6.0 ***
Hello World! frdm_mcxn236

```

✓ Checkpoint A

Screenshot of hello_world running on the board.

? Question A1

west update fetches many modules (HALs, mbedTLS, LVGL, etc.). Which module provides the NXP MCX HAL for the FRDM-MCXN236? (Hint: look in zephyrproject/modules/hal/.)

6. Part B — Port the Producer-Consumer Pipeline

6.1. Step 1 — FreeRTOS → Zephyr API Mapping

Complete this mapping table before writing any code:

FreeRTOS (Lab 03)**Zephyr equivalent**

```

xTaskCreate(fn, name, stack_words,
...)
K_THREAD_DEFINE(tid, stack_bytes,
fn, ...)
xQueueCreate(depth, item_size)
xQueueSend(q, &item, timeout)
xQueueReceive(q, &item,
portMAX_DELAY)
xSemaphoreCreateBinary()
xSemaphoreGive(sem)
xSemaphoreTake(sem, portMAX_DELAY)
vTaskDelay(pdMS_TO_TICKS(n))
vTaskDelayUntil(&last, period)

```

6.2. Step 2 — prj.conf

```

# prj.conf
CONFIG_SERIAL=y
CONFIG_LOG=y
CONFIG_LOG_DEFAULT_LEVEL=3

# Threading
CONFIG_MAIN_STACK_SIZE=2048
CONFIG_HEAP_MEM_POOL_SIZE=4096

# No float printf -- save flash
CONFIG_CBPRINTF_FP_SUPPORT=n

```

6.3. Step 3 — CMakeLists.txt

```

cmake_minimum_required(VERSION 3.20)
find_package(Zephyr REQUIRED HINTS $ENV{ZEPHYR_BASE})
project(lab11_zephyr)

target_sources(app PRIVATE
    src/main.c
    src/sensor_thread.c
    src/logger_thread.c
)

```

6.4. Step 4 — Sensor Producer Thread

In src/sensor_thread.c:

```

1 #include <zephyr/kernel.h>
2 #include "sensor_thread.h"
3
4 /* Message queue: 10 items of SensorReading_t */
5 K_MSGQ_DEFINE(sensor_msgq, sizeof(SensorReading_t), 10, 4);
6
7 static void sensor_thread_fn(void *a, void *b, void *c)
8 {
9     SensorReading_t reading = { .sensor_id = 1, .value = 0 };
10
11     while (1) {

```

```

12     k_sleep(K_MSEC(100));
13     reading.value = (reading.value + 7) % 1000;
14     reading.ts_ms = k_uptime_get_32();
15
16     if (k_msgq_put(&sensor_msgq, &reading, K_NO_WAIT) != 0)
17         LOG_WRN("queue full -- dropped");
18 }
19 }
20
21 K_THREAD_DEFINE(sensor_tid, 1024, sensor_thread_fn,
22                 NULL, NULL, NULL, 5, 0, 0);

```

6.5. Step 5 — Logger Consumer Thread

In `src/logger_thread.c`:

```

1  #include <zephyr/kernel.h>
2  #include <zephyr/logging/log.h>
3  #include "sensor_thread.h" /* for sensor_msgq extern */
4
5  LOG_MODULE_REGISTER(logger, LOG_LEVEL_INF);
6
7  static void logger_thread_fn(void *a, void *b, void *c)
8  {
9      SensorReading_t r;
10     uint32_t count = 0;
11     while (1) {
12         k_msgq_get(&sensor_msgq, &r, K_FOREVER);
13         count++;
14         LOG_INF("#%u sensor=%d val=%d t=%u ms",
15               count, r.sensor_id, r.value, r.ts_ms);
16     }
17 }
18
19 K_THREAD_DEFINE(logger_tid, 1024, logger_thread_fn,
20                 NULL, NULL, NULL, 3, 0, 0);

```

6.6. Step 6 — Build and Flash

```

cd ~/zephyrproject/lab11_zephyr
west build -b frdm_mcxn236 .
west flash

```

Expected UART output:

```

[00:00:00.012,000] <inf> logger: #1  sensor=1  val=7  t=100 ms
[00:00:00.112,000] <inf> logger: #2  sensor=1  val=14 t=200 ms
...

```

✓ Checkpoint B

Terminal showing Zephyr logging output from the ported pipeline.

? Question B1

In FreeRTOS, `xQueueSend` with `portMAX_DELAY` blocks the calling task forever. What is the Zephyr equivalent timeout value for “wait forever”?

? Question B2

K_THREAD_DEFINE takes stack size in **bytes** while FreeRTOS xTaskCreate takes it in **words** (4 bytes each). Convert the Lab 03 sensor task's 128-word stack to the correct Zephyr byte value.

7. Part C — Device Tree Sensor (BME280)**7.1. Step 1 — Write the Overlay**

Create app.overlay in the project root:

```
/* app.overlay -- add BME280 sensor on LPI2C1 */
&lpi2c1 {
    status = "okay";
    clock-frequency = <I2C_BITRATE_STANDARD>;
    pinctrl-0 = <&pinmux_lpi2c1>;
    pinctrl-names = "default";

    bme280: bme280@76 {
        compatible = "bosch,bme280";
        reg = <0x76>;
        status = "okay";
    };
};
```

Add to prj.conf:

```
CONFIG_I2C=y
CONFIG_SENSOR=y
CONFIG_BME280=y
```

7.2. Step 2 — Use the Sensor API

Add a BME280 reading thread in src/sensor_thread.c:

```
1 #include <zephyr/drivers/sensor.h>
2
3 static const struct device *bme280 =
4     DEVICE_DT_GET(DT_NODELABEL(bme280));
5
6 static void bme_thread_fn(void *a, void *b, void *c)
7 {
8     if (!device_is_ready(bme280)) {
9         LOG_ERR("BME280 not ready");
10        return;
11    }
12
13    struct sensor_value temp, pressure, humidity;
14    while (1) {
15        k_sleep(K_MSEC(1000));
16        sensor_sample_fetch(bme280);
17        sensor_channel_get(bme280, SENSOR_CHAN_AMBIENT_TEMP, &temp);
18        sensor_channel_get(bme280, SENSOR_CHAN_PRESS, &pressure);
19        sensor_channel_get(bme280, SENSOR_CHAN_HUMIDITY, &humidity);
20        LOG_INF("T=%.2f C P=%.2f hPa H=%.2f %%",
21                sensor_value_to_double(&temp),
22                sensor_value_to_double(&pressure) / 1000.0,
23                sensor_value_to_double(&humidity));
24    }
25 }
26
```

```
27 K_THREAD_DEFINE(bme_tid, 1024, bme_thread_fn, NULL, NULL, NULL, 4,
    0, 0);
```

Note

If you do not have a physical BME280, use the Zephyr emulator by adding to `prj.conf`:

```
CONFIG_BME280_TRIGGER_NONE=y
CONFIG_EMUL=y
CONFIG_BME280_EMUL=y
```

The emulator returns synthetic values, letting you exercise the full API without hardware.

Checkpoint C

Terminal showing BME280 (or emulated) temperature/pressure/humidity readings every second.

Question C1

Where in the Zephyr source tree is the BME280 driver located? What is the value of its `compatible` string in the DTS binding file?

Question C2

You want to add a second BME280 at I2C address 0x77 on the same bus. What change do you make — in `app.overlay` only, or also in C code? Why?

8. Part D — Binary Size Comparison

8.1. Step 1 — Measure Zephyr Binary Size

```
west build -b frdm_mcxn236 .
arm-none-eabi-size build/zephyr/zephyr.elf
```

8.2. Step 2 — Measure Equivalent FreeRTOS Binary Size

```
cd ../lab03_aperiodic_server
mingw32-make
arm-none-eabi-size lab03.elf
```

8.3. Step 3 — Complete the Comparison Table

Metric	FreeRTOS (Lab 03)	Zephyr (Lab 11)
.text (code)		
.rodata (const)		
.data + .bss		
Total flash		
Build time (cold)		
Lines of config		

Question D1

Zephyr is typically larger than FreeRTOS for the same feature set. Where does most of the extra flash usage come from? (Hint: run `west build -t rom_report`.)

? Question D2

You need to add BLE advertising to this application. Describe how you would do this in Zephyr (which `prj.conf` options, which API) vs. how you would approach it with FreeRTOS (which external library, which integration steps). Which is faster to integrate and why?

9. Part E — Kconfig Exploration (Optional Challenge)

```
west build -b frdm_mcxn236 -t menuconfig .
```

Navigate to:

- Kernel → Threads — find `CONFIG_MAIN_STACK_SIZE`
- Subsystems → Logging — observe log level options and backend choices
- Drivers → Sensor — note available sensor drivers

? Question Challenge

Disable the shell (`CONFIG_SHELL=n`) and the logging backend to serial (`CONFIG_LOG_BACKEND_UART=n`). Rebuild and measure the flash savings. How much code do logging and the shell add to a Zephyr application?

10. Deliverables

#	Item	Details
1	Part A screenshot	<code>hello_world</code> running on the board
2	Part B API mapping table	Completed FreeRTOS → Zephyr column
3	Part B terminal	Zephyr producer-consumer output
4	Part C terminal	BME280 (real or emulated) sensor readings
5	Part D table	Binary size comparison
6	File listings	<code>prj.conf</code> and <code>app.overlay</code>
7	Written answers	Questions A1, B1–B2, C1–C2, D1–D2
8	Part E (optional)	Flash savings measurement

11. Key Concepts Demonstrated

Concept	Where you saw it
west workspace init and build	Part A
Kconfig compile-time configuration	Part B <code>prj.conf</code>
<code>K_THREAD_DEFINE</code> and <code>K_MSGQ_DEFINE</code>	Part B porting
Device Tree overlay for I2C sensor	Part C
Unified Sensor API	Part C
FreeRTOS vs. Zephyr binary size	Part D

12. Reference

Resource	Relevant sections
Zephyr documentation	Getting Started, Threads, Message Queue, Sensor API
Zephyr Device Tree	boards/arm/frdm_mcxn236/, dts/bindings/sensor/bosch,bme280.yaml
Nordic DevAcademy	NCS Fundamentals — Device Tree module
FreeRTOS docs	xQueueCreate (for comparison)
